

COMPUTER SCIENCE TRIPOS Part IB – 2023 – Paper 5

7 Introduction to Computer Architecture (swm11)

Eight great ideas
in computer
architecture

(a) What is Amdahl's law?

[2 marks]

Answer: Amdahl's law for sequential programs can be stated as:

$$\text{execution time after improvement} = \frac{\text{execution time affected by improvement}}{\text{amount of improvement}} + \text{execution time unaffected}$$

Which can also be expressed as:

$$s_{\text{improvement}} = \frac{1}{(1-p) + \frac{p}{s}}$$

where:

- $s_{\text{improvement}}$ speedup after improvement
- s is the speedup of the part of the task that benefits from some improvement
- p is the proportion of execution time that the improvement can be used

Amdahl's law can also be applied to parallel programs by looking at the ratio of sequential code to that run in parallel.

Reduced
instruction set
computers and
RISC-V;
Pipelining

(b) For a 64-bit RISC-V pipelined processor, how could the following code be optimised to reduce execution time? Justify your answer. [4 marks]

```
lw t0, 0(a0)    # load 32-bits from address a0 into register t0
sw t0, 0(a1)    # store 32-bits to address a0 from register t0
lw t1, 4(a0)
sw t1, 4(a1)
lw t2, 8(a0)
sw t2, 8(a1)
lw t3, 12(a0)
sw t3, 12(a1)
```

Answer: There are two optimisations that can be applied to this code:

- To avoid load-to-use penalties in the pipeline as a result of the pipeline stalling to avoid data hazards, we can hoist the loads above the stores.
- If we can assume that $a0$ and $a1$ are double word aligned, we can halve the number of load and store instructions by accessing doubles rather than words at a time.

The resulting code would be:

```
ld t0, 0(a0)
ld t1, 8(a0)
sd t0, 0(a1)
sd t1, 8(a1)
```

Reduced
instruction set
computers and
RISC-V;
Pipelining

(c) A new RISC-V R-type instruction `swap` is proposed to swap two registers. For example:

```
swap t0, t1
```

would swap the contents of registers `t0` and `t1`. What would be the challenges in implementing this proposed `swap` instruction? [3 marks]

Answer: The biggest challenge to implement the `swap` instruction is the need to write to two registers simultaneously. No other RISC-V instruction writes to two registers, so this would add a lot of complexity to the register file.

Reduced
instruction set
computers and
RISC-V

- (d) A counterproposal is to introduce `swap` as a pseudo instruction that unpacks to a short sequence of real instructions. What sequence of instructions could be generated that swaps two registers without using an additional register? [*Hint:* it involves using `xor`] [3 marks]

Answer:

```
xor t0, t1, t0    # t0' = t1 ^ t0
xor t1, t0, t1    # t1' = t0 ^ t1' = t0 ^ t1 ^ t1 = t0
xor t0, t1, t0    # t0'' = t1' ^ t0' = t0 ^ t1 ^ t0 = t1
```

Multicore
Processors

- (e) RISC-V provides an atomic swap in memory operation `amoswapd` that takes an address held in a register (e.g. `a0`), a source register (e.g. `t0`), a destination register (e.g. `t1`). For these example register allocations, the operation performs an atomic read-modify-write where the value at address `a0` is stored in register `t1` and the value in register `t0` is written to address `a0`. How could the same operation be achieved using load-reserved (`lr`) and store-conditional (`sc`) instructions instead of using `amoswapd`? Explain your code by commenting it. [4 marks]

Answer:

```
retry: mv t3, t0          # t3=t0, i.e. make a copy of t0
      lr t1, 0(a0)        # load-reserved from address a0
      sc t3, 0(a0)        # store-conditional t3 to address a0
      bnez t3, retry      # retry operation if sc fails (r3!=0)
```

Note that the lecture slides incorrectly said that `sc` returns 0 on failure and 1 on success, but it should be 0 on success and non-zero on failure. So I'll accept either possibility so long as the code is explained.

Multicore
Processors

- (f) If a computer system uses the MSI cache coherence protocol, and if multiple processor cores are reading shared data at address *a* at the same time, what happens when one processor core writes a word to address *a*? [4 marks]

Answer: Given that data at address *a* is being read by multiple cores, the cachelines will be in the S (shared) state. For a processor core to write to address *a*, all other caches must first be invalidated (I-state) at address *a* using a read-exclusive bus operation. Then the cacheline held by the writing core can be placed into the M (modified) state and the write operation performed. This guarantees that the write is atomic since no other processor core can modify

— *Solution notes* —

address a at the same time. Once the write is complete, if other cores wish to read from address a they will send a read bus operation, placing the cacheline back into the S (shared) state.
